HIGH-LEVEL DESIGN (HLD)

Project Title:

RAG-Based Customer Support Assistant using LangGraph with HITL

INTRODUCTION

In today's digital environment, customer support systems are expected to deliver fast, accurate, and consistent responses. Traditional support methods often rely heavily on human agents, leading to delays, increased operational costs, and inconsistent information delivery. With the growing availability of large volumes of documentation such as FAQs, manuals, and policy documents, there is a need for intelligent systems that can efficiently retrieve and present relevant information to users.

This project presents the design of a **Retrieval-Augmented Generation (RAG) based Customer Support Assistant**. The system combines information retrieval techniques with the generative capabilities of Large Language Models (LLMs) to provide context-aware and accurate responses. Instead of generating answers solely based on pre-trained knowledge, the system retrieves relevant information from a given PDF knowledge base and uses it to produce more reliable outputs.

The solution incorporates a **graph-based workflow using LangGraph**, enabling structured control over the query processing pipeline and supporting conditional decision-making. Additionally, a **Human-in-the-Loop (HITL)** mechanism is integrated to handle cases where the system lacks confidence or encounters complex queries, ensuring reliability and improved user trust.

The objective of this system is to design a scalable, modular, and intelligent customer support assistant that demonstrates the application of modern AI techniques such as embeddings, vector databases, retrieval systems, and workflow orchestration.

SYSTEM OVERVIEW

a) Problem Statement

Customer support systems often face challenges such as slow response times, inconsistent answers, and a high dependency on human agents, which can impact overall efficiency and user satisfaction. With the increasing availability of digital documentation, there is a growing need for intelligent systems that can quickly access and utilize this information. This project aims to develop a smart customer support assistant that can retrieve relevant information from documents, generate accurate and context-aware responses, and intelligently escalate complex or uncertain queries to human agents when necessary, thereby improving both efficiency and reliability of support services.

b) Objective

To design a **RAG-based customer support assistant** that:

- Processes PDF documents
- Uses embeddings for retrieval
- Generates contextual responses using LLM
- Uses a graph-based workflow (LangGraph)
- Supports Human-in-the-Loop (HITL) escalation

c) Scope

Included:

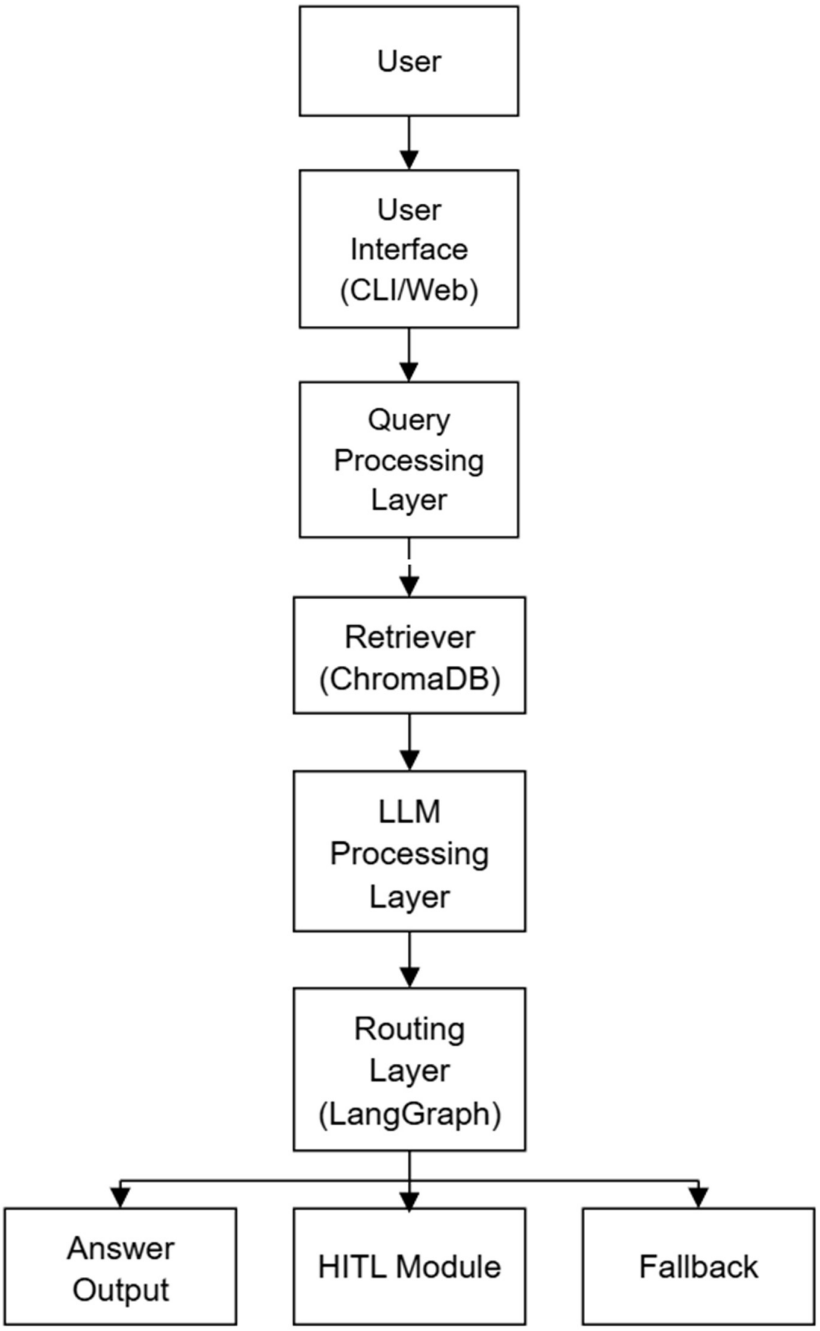
- PDF-based knowledge system
- Query answering using RAG
- Conditional routing
- HITL escalation

Not Included:

- Multi-user authentication
- Real-time deployment
- Voice-based interaction

SYSTEM ARCHITECTURE

a) Architecture Overview



b) Key Components

User Interface:

The User Interface serves as the interaction layer between the user and the system. It allows users to input their queries in a simple and intuitive manner and displays the generated responses clearly, ensuring a smooth and user-friendly experience.

Document Ingestion Pipeline:

The Document Ingestion Pipeline is responsible for processing the input knowledge base. It loads PDF documents, extracts the textual content, and converts it into a structured format suitable for further processing such as chunking and embedding.

Chunking System:

The Chunking System is responsible for dividing the extracted text into smaller, manageable segments. This process ensures that the information is structured in a way that improves retrieval accuracy and allows the system to efficiently handle large documents.

Embedding System:

The Embedding System converts textual data into numerical vector representations. These embeddings capture the semantic meaning of the text, enabling the system to understand context and perform similarity-based searches effectively.

Vector Database (ChromaDB):

The Vector Database stores the generated embeddings along with their associated text chunks. It enables fast and efficient similarity search, allowing the system to quickly identify the most relevant information for a given query.

Retrieval Layer:

The Retrieval Layer is responsible for fetching the most relevant text chunks from the vector database based on the user query. It uses similarity metrics to identify and return the top-k matching results, which are then used for response generation.

LLM Processing Layer

The LLM Processing Layer is responsible for generating the final response to the user. It combines the user query with the retrieved contextual information and uses a Large Language Model to produce accurate, coherent, and context-aware answers.

Workflow Orchestration (LangGraph)

The Workflow Orchestration layer, implemented using LangGraph, manages the overall execution flow of the system. It enables structured and stateful processing, allowing the system to make dynamic decisions and follow different paths based on the query and intermediate results.

Routing Layer

The Routing Layer is responsible for making decisions based on the output of the LLM and system confidence. It determines whether the generated response is sufficient to be delivered to the user or if the query should be escalated for further handling.

Human-in-the-Loop (HITL) Module

The HITL Module handles cases where the system is unable to confidently generate an appropriate response. In such scenarios, the query is escalated to a human agent, and the human-generated response is then provided back to the user, ensuring reliability and accuracy.

DATA FLOW

a) Document Processing Flow

1. Load PDF
2. Extract text
3. Split into chunks
4. Generate embeddings
5. Store in ChromaDB

b) Query Processing Flow

1. User submits query
2. Query converted to embedding
3. Retrieve relevant chunks
4. Pass to LLM
5. Generate response
6. Evaluate confidence
7. Route:

- High confidence - Answer
- Low confidence - HITL

TECHNOLOGY CHOICES

ChromaDB

ChromaDB is a lightweight and easy-to-integrate vector database used for storing and managing embeddings generated from text data. It supports efficient similarity search operations, allowing the system to quickly retrieve relevant information based on semantic similarity. Its simplicity and compatibility with modern AI frameworks make it a suitable choice for building retrieval-based applications.

LangGraph

LangGraph is used to design and manage the workflow of the system through a graph-based architecture. It enables structured, stateful execution of tasks and supports conditional routing based on intermediate results. This allows the system to dynamically decide the flow of operations, making it more flexible and suitable for complex decision-making scenarios.

LLM (e.g., OpenAI / Mistral)

The Large Language Model plays a crucial role in generating responses by combining the user query with the retrieved contextual information. It is capable of producing human-like, coherent, and context-aware answers. By leveraging advanced natural language understanding, the LLM enhances the overall quality and relevance of the system's responses.

LangChain

LangChain acts as a supporting framework that simplifies the integration of various components within the system. It provides tools and utilities for document loading, text splitting, embedding generation, and retrieval processes. This reduces development complexity and enables faster implementation of the RAG pipeline.

SCALABILITY CONSIDERATIONS

Handling Large Documents

As the size of input documents increases, efficient processing becomes critical for maintaining system performance. The system addresses this by implementing an effective chunking strategy, where large documents are divided into smaller, manageable segments. This ensures that each chunk remains contextually meaningful while also being suitable for embedding generation and retrieval. Additionally, all generated embeddings are indexed and stored in a vector database, allowing for fast and scalable similarity search even when dealing with extensive knowledge bases. This approach ensures that the system can handle large volumes of data without significant degradation in performance.

Handling High Query Load

To ensure the system performs reliably under high user traffic, several optimization strategies are considered. Frequently asked queries can be cached along with their corresponding responses, reducing the need for repeated retrieval and generation processes. This significantly improves response time for common queries. Furthermore, the system can be designed to support asynchronous processing, allowing multiple queries to be handled concurrently without blocking system resources. These techniques help maintain system responsiveness and scalability as the number of users increases.

Latency Optimization

Minimizing response time is essential for providing a smooth user experience. One approach to reducing latency is optimizing the chunk size used during document processing. Smaller chunks can improve retrieval speed, while carefully chosen overlap ensures that contextual information is not lost. Additionally, retrieval strategies such as limiting the number of returned chunks (top-k selection) and using efficient similarity search algorithms can further reduce processing time. Optimizing LLM usage, such as controlling prompt size and reducing unnecessary context, also contributes to faster response generation.

System Scalability and Future Expansion

The system is designed with modular components, allowing easy scaling and future enhancements. As data volume grows, the vector database can be expanded or replaced with more advanced distributed solutions. Similarly, the architecture allows integration with more powerful embedding models or LLMs without major redesign. The workflow orchestration using LangGraph ensures that additional nodes or decision paths can be introduced as needed, enabling the system to evolve with increasing complexity and requirements.

ASSUMPTIONS

The design of the proposed RAG-based customer support assistant is based on several assumptions that define the expected operating conditions of the system. Firstly, it is assumed that the input documents provided to the system are well-structured and primarily in PDF format. This ensures that the text extraction process can be performed effectively without significant loss of information or formatting inconsistencies. Properly structured documents improve the quality of chunking and, consequently, the accuracy of retrieval.

It is also assumed that the system has access to reliable internet connectivity, especially when using cloud-based Large Language Model (LLM) APIs for generating responses. Since LLMs play a critical role in producing context-aware answers, uninterrupted connectivity is essential for smooth system operation.

Another key assumption is that users will primarily ask domain-specific queries related to the content of the uploaded documents. The system is designed to retrieve and generate responses based on the available knowledge base, and therefore performs best when queries are relevant to the provided data. It is also assumed that the volume of data and query load will remain within manageable limits for the chosen tools and technologies, such as ChromaDB and LangGraph.

Additionally, it is assumed that the embedding model used is capable of capturing the semantic meaning of the text effectively, and that the similarity search mechanism will retrieve contextually relevant chunks. The system also assumes that a predefined threshold can be used to determine confidence levels for routing decisions, including escalation to the Human-in-the-Loop (HITL) module.

LIMITATIONS

Despite its capabilities, the system has certain limitations that must be acknowledged. One of the primary limitations is its performance with ambiguous or poorly defined queries. If a user query lacks clarity or context, the retrieval system may not be able to identify the most relevant information, leading to less accurate or incomplete responses.

The effectiveness of the system is highly dependent on the quality and completeness of the input documents. If the knowledge base contains outdated, incomplete, or poorly formatted information, the generated responses may also reflect these shortcomings. This highlights the importance of maintaining high-quality and well-organized documentation.

Another limitation arises from the inherent behavior of Large Language Models. While LLMs are powerful, their responses may sometimes vary for the same input due to probabilistic generation. In some cases, the model may produce responses that are partially incorrect or not fully aligned with the retrieved context, especially if the prompt is not well-structured.

The system is also limited in handling highly complex, multi-step reasoning queries or queries that require external knowledge beyond the provided documents. In such scenarios, the HITL mechanism becomes essential, but it also introduces dependency on human availability.

Furthermore, the current design does not account for real-time updates to the knowledge base or continuous learning from user interactions. Scalability may also become a challenge if the system is deployed for large-scale, real-time applications without further optimization or distributed infrastructure.

Overall, while the system provides a strong foundation for intelligent customer support, these limitations highlight areas for future improvement and enhancement.